

Jawaban Modul Praktikum

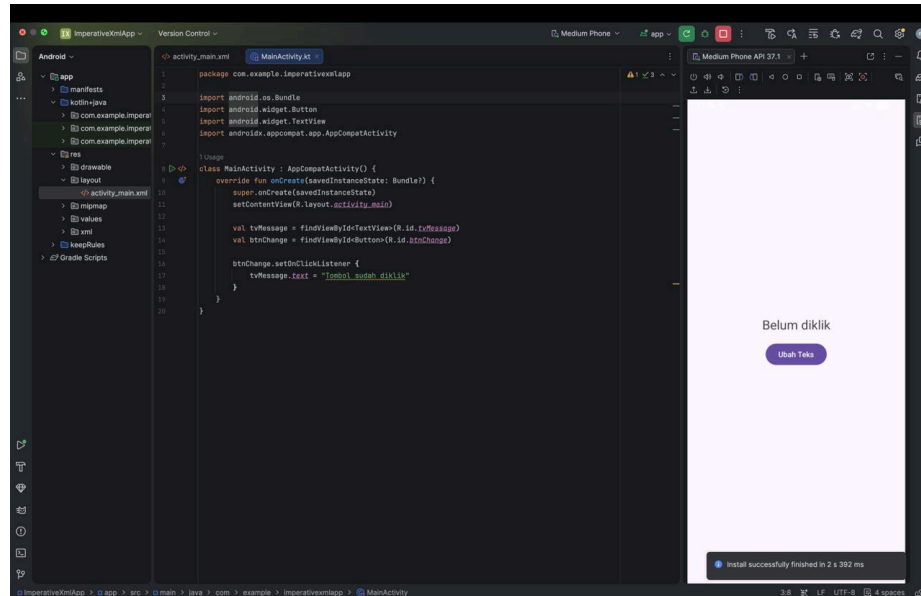
Event, State, dan Implisit Intent pada Jetpack Compose

Nama: Muhammad Abdillah
NIM: 24515040111015

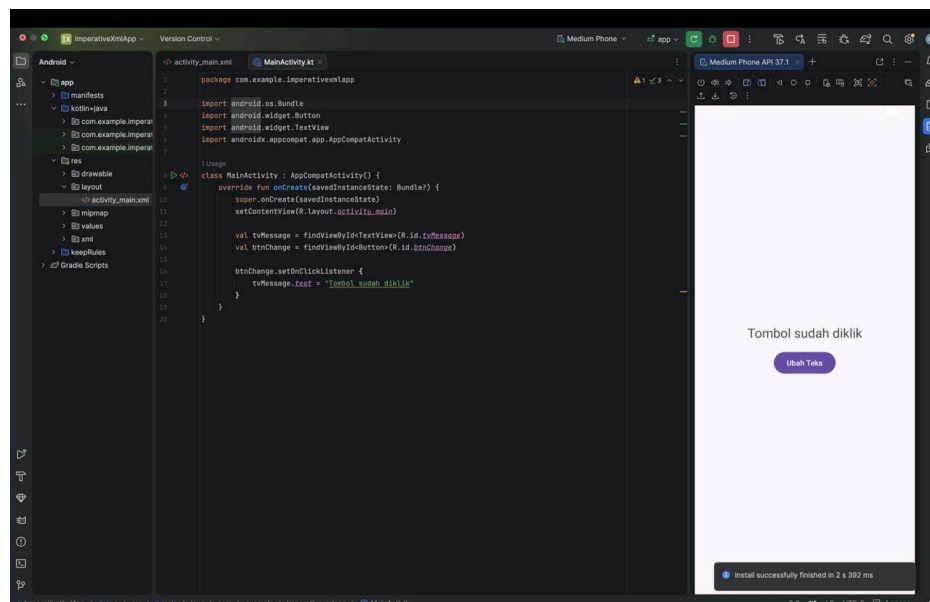
Percobaan 1 : Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik.

Sebelum :



Sesudah :



2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.

Objek yang diubah secara langsung adalah TextView bernama tvMessage, tepatnya properti text miliknya.

Baris kode: `tvMessage.text = "Tombol sudah diklik"`

3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.

Yang diubah adalah nilai state message (bertipe String, dibungkus mutableStateOf dan remember), bukan objek UI-nya secara langsung.

Baris kode: `message = "Tombol sudah diklik"`

4. Mengapa pada Compose tidak terdapat perintah seperti `tvMessage.text` atau `setText()`?

Karena Compose menganut pendekatan deklaratif: composable Text tidak dipegang/direferensikan secara manual oleh programmer, melainkan hanya "menggambarkan" bahwa isinya berasal dari state message ($UI = f(state)$). Ketika message berubah, Compose sendiri yang mendeteksi perubahan itu dan menjalankan recomposition untuk menggambar ulang Text dengan nilai terbaru. Programmer tidak perlu (dan tidak bisa) memerintah komponen UI secara langsung seperti pada View/XML.

5. Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.

Event (klik tombol `btnChange`) memicu listener `onClickListener` -> program mengambil referensi objek View (`TextView tvMessage`) melalui `findViewById()` -> program secara eksplisit mengubah properti objek tersebut (`tvMessage.text`) -> tampilan berubah karena properti View sudah diubah langsung.

6. Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

Event (klik tombol) dijalankan pada lambda `onClick` -> nilai state message diubah -> Compose runtime mendeteksi bahwa state yang sedang diamati (diobservasi) oleh composable Text telah berubah -> Compose menjalankan recomposition, yaitu menjalankan ulang fungsi composable yang bergantung pada state tersebut -> tampilan Text diperbarui menampilkan nilai message yang baru.

7. Apa yang terjadi jika message diganti menjadi variabel biasa: `var message = "Belum diklik"`? Jelaskan penyebabnya.

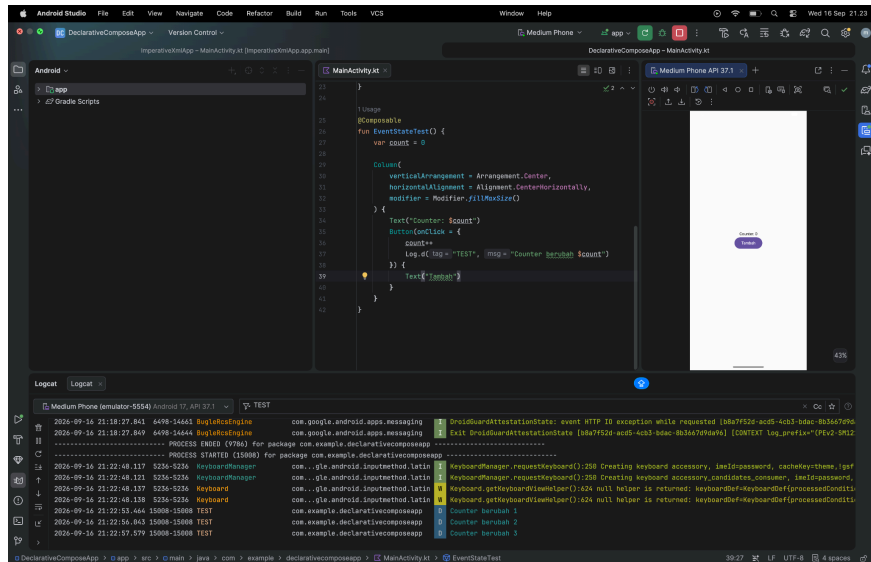
Nilai message di memori akan tetap berubah menjadi "Tombol sudah diklik" saat tombol diklik, tetapi tampilan pada layar tidak akan berubah dan tetap menampilkan "Belum diklik". Penyebabnya, variabel biasa (bukan `mutableStateOf`) tidak diamati (observable) oleh Compose, sehingga perubahan nilainya tidak memicu recomposition. Compose hanya akan menggambar ulang UI jika ada state yang dapat diamati yang berubah; variabel biasa hanya akan terbaca ulang jika recomposition terjadi karena sebab lain (misalnya rotasi layar atau perubahan state lain).

8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

Pada pendekatan imperatif, programmer berpikir dengan cara "cari komponen UI-nya, lalu perintahkan ia berubah" — programmer bertanggung jawab penuh menentukan komponen mana yang diperbarui dan bagaimana caranya. Pada pendekatan deklaratif, programmer cukup berpikir "seperti apa seharusnya tampilan ini berdasarkan data/state saat ini", tanpa perlu memerintah komponen secara manual; framework (Compose) yang akan menentukan dan menjalankan pembaruan tampilan setiap kali state yang relevan berubah.

Percobaan 2 : Event Handling State

1. Jalankan program (versi variabel biasa `var count = 0`), klik tombol "Tambah" beberapa kali, ambil screenshot tampilannya.



2. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

Muncul log baru dengan tag TEST yang bertuliskan "Counter berubah : X", di mana X adalah nilai count yang terus bertambah (1, 2, 3, dst.) setiap kali tombol diklik.

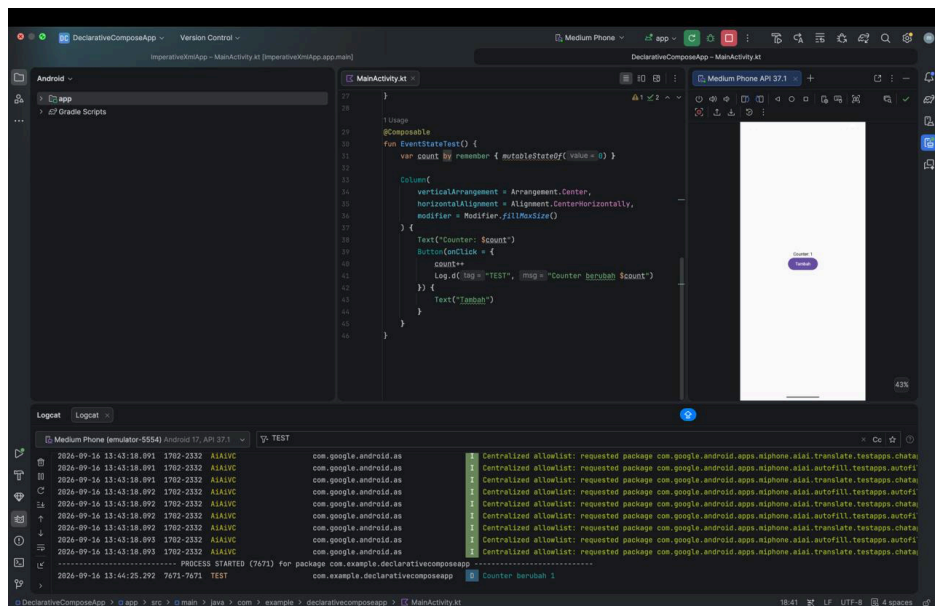
3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Tidak terjadi perubahan apa pun pada tampilan. Teks "Counter : 0" tetap menampilkan angka 0 meskipun tombol sudah diklik berkali-kali.

4. Apa yang menyebabkan hal tersebut terjadi?

Variabel count dideklarasikan sebagai variabel biasa (var count = 0), bukan sebagai state Compose (mutableStateOf). Compose tidak mengamati perubahan pada variabel biasa, sehingga meskipun nilainya bertambah di memori (terbukti dari LogCat), Compose tidak mengetahui bahwa ada perubahan yang perlu digambar ulang, sehingga tidak terjadi recomposition dan tampilan tidak diperbarui.

8. Jalankan program kembali (setelah count diubah menjadi state), klik tombol "Tambah" beberapa kali, ambil screenshot tampilannya.



9. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

Sama seperti sebelumnya, muncul log dengan tag TEST bertuliskan "Counter berubah : X" dengan nilai X yang terus bertambah setiap klik.

10. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Kali ini tampilan ikut berubah: angka pada teks "Counter : X" bertambah sesuai jumlah klik, sinkron dengan nilai yang tercatat di LogCat.

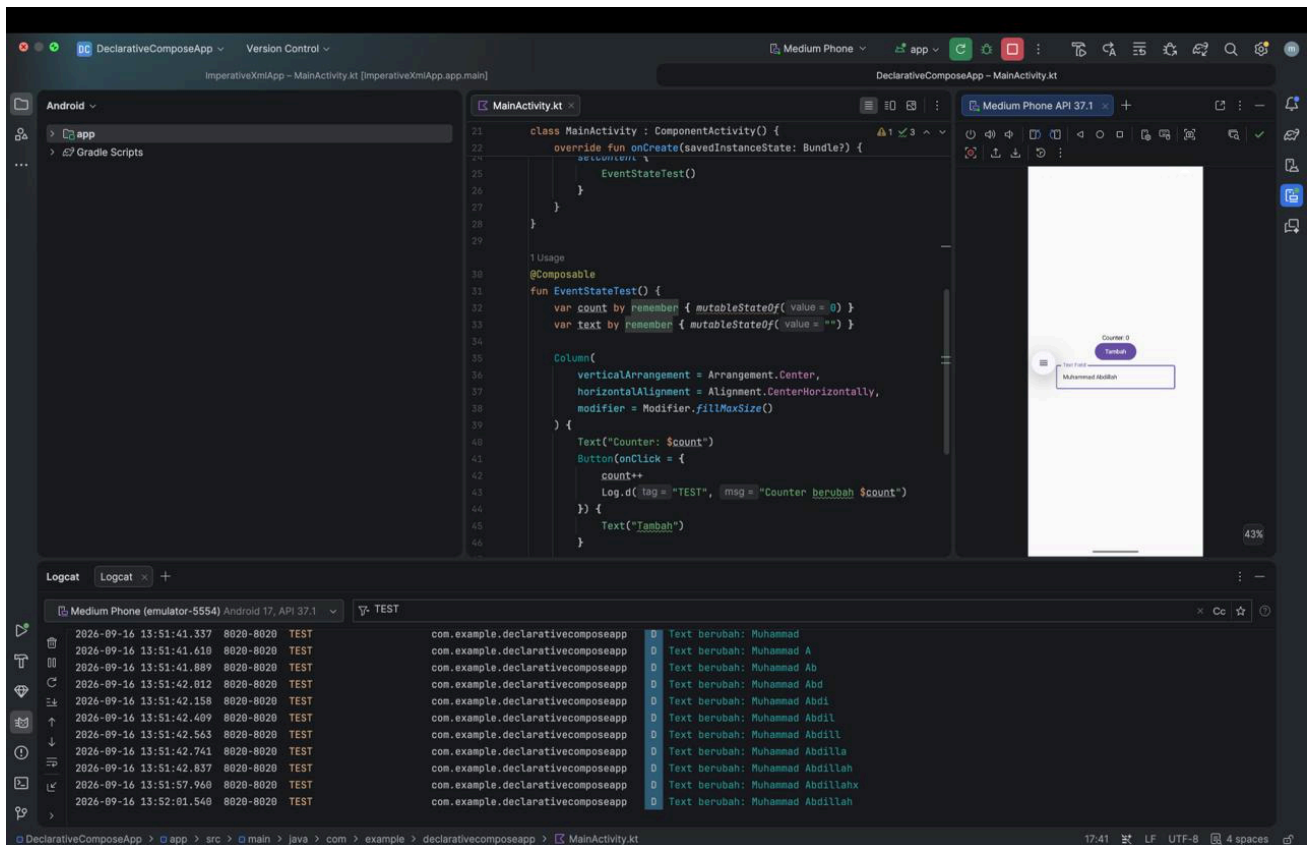
11. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

Karena count sekarang berupa state Compose (dibungkus mutableStateOf dan remember, dengan property delegation by). Compose dapat mengamati perubahan pada state ini, sehingga setiap kali count berubah, Compose otomatis menjalankan recomposition pada bagian UI (Text) yang membaca nilai count, sehingga tampilan ikut diperbarui — berbeda dengan percobaan sebelumnya yang menggunakan variabel biasa dan tidak diamati oleh Compose.

12. Apa yang dimaksud recomposition pada Compose?

Recomposition adalah proses Compose menjalankan ulang (memanggil ulang) fungsi-fungsi composable yang bergantung pada suatu state, setiap kali state tersebut berubah, agar tampilan yang dihasilkan selalu sesuai/merepresentasikan data (state) terbaru. Compose bersifat pintar karena hanya melakukan recomposition pada bagian UI yang benar-benar terpengaruh oleh perubahan state, bukan menggambar ulang seluruh layar.

14. Jalankan program kembali (setelah menambahkan state text dan OutlinedTextField), klik tombol "Tambah" beberapa kali dan ketik pada Text Field, ambil screenshot tampilannya.



15. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?

Muncul log baru dengan tag TEST bertuliskan "Text berubah : <isi_text_saat_ini>", yang menampilkan nilai terbaru dari text setiap kali ada perubahan (setiap huruf yang diketik atau dihapus).

16. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?

Tampilan OutlinedTextField langsung menampilkan huruf yang baru saja diketik, sehingga isi kotak teks selalu sesuai dengan apa yang diketik pengguna secara real-time.

17. Apa yang menyebabkan hal tersebut terjadi?

Karena value pada `OutlinedTextField` diikat (bound) ke state `text`, dan setiap kali pengguna mengetik, callback `onValueChange` dipanggil dan mengubah nilai state `text` tersebut. Perubahan pada state `text` ini memicu recomposition pada `OutlinedTextField`, sehingga tampilannya diperbarui sesuai nilai `text` yang baru — pola ini disebut `state hoisting/controlled component` pada `Compose`.

18. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?

1. `count` — state bertipe `Int` (`mutableStateOf(0)`), merepresentasikan nilai counter.
2. `text` — state bertipe `String` (`mutableStateOf("")`), merepresentasikan isi `Text Field`.

19. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

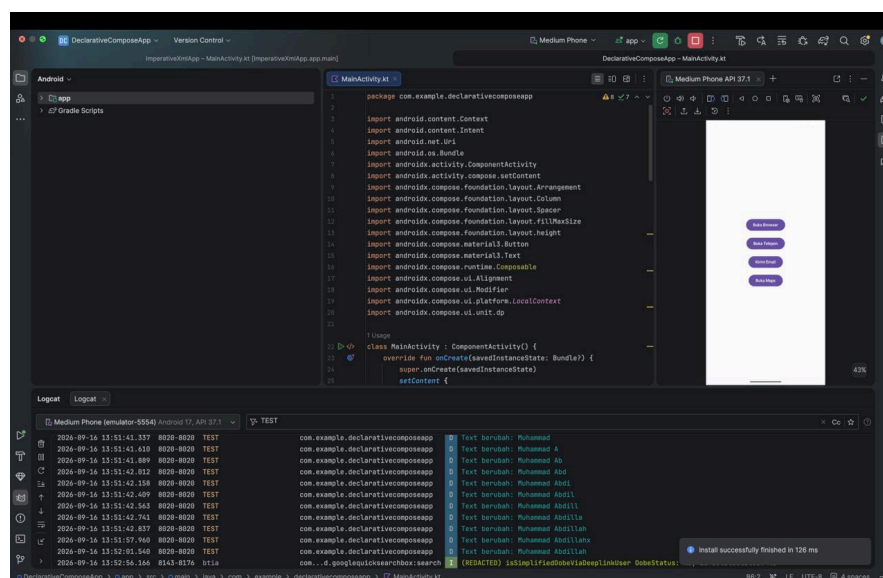
1. `onClick` pada `Button` "Tambah" — event ketika tombol ditekan.
2. `onValueChange` pada `OutlinedTextField` — event ketika isi teks pada `Text Field` berubah (setiap ketikan/penghapusan huruf).

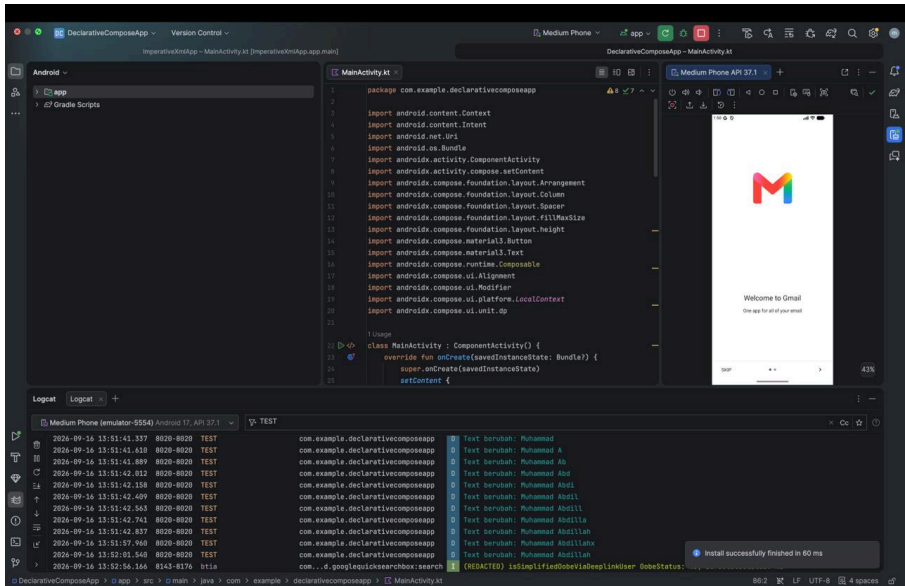
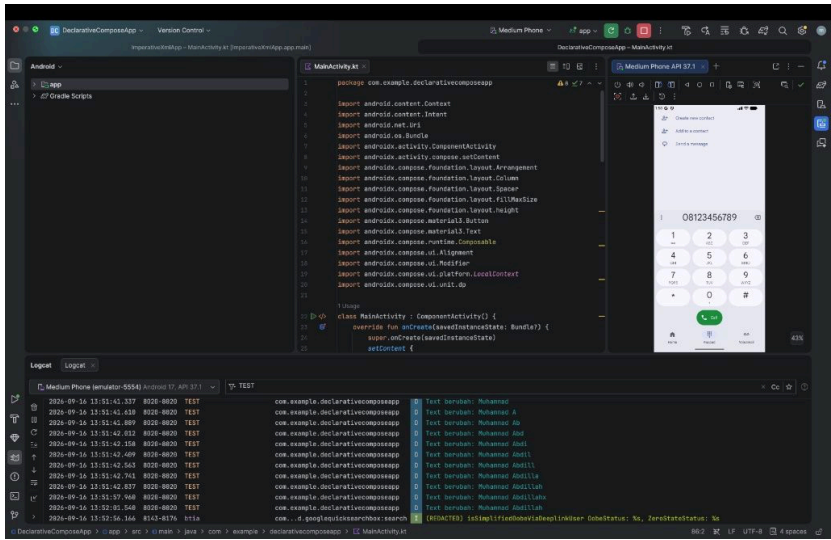
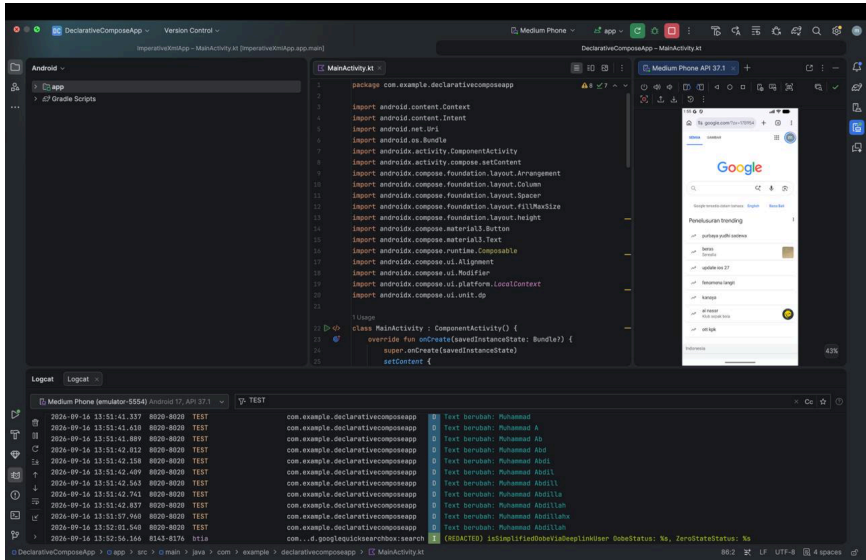
20. Kapan event-event tersebut dijalankan?

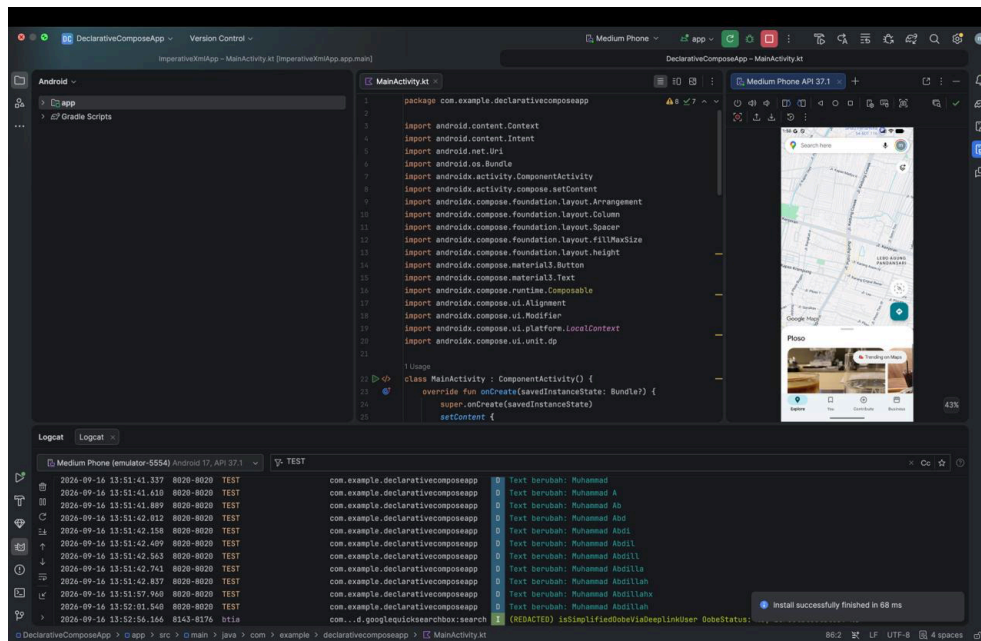
Event `onClick` dijalankan setiap kali pengguna menekan (mengetuk) tombol "Tambah" di layar. Event `onValueChange` dijalankan setiap kali nilai teks pada `OutlinedTextField` berubah, yaitu setiap kali pengguna mengetik atau menghapus karakter pada kolom input tersebut.

Percobaan 3 : Implisit Intent

1-6. Jalankan program, uji setiap tombol (Buka Browser, Buka Telepon, Kirim Email, Buka Maps), amati hasilnya, dan ambil screenshot.







11. Apa perbedaan fungsi dari masing-masing action intent tersebut (ACTION_VIEW, ACTION_DIAL, ACTION_SENDTO)?

Intent.ACTION_VIEW — menampilkan/membuka data kepada pengguna menggunakan aplikasi yang sesuai dengan jenis datanya (misalnya membuka URL di browser, atau membuka koordinat lokasi di aplikasi peta).

Intent.ACTION_DIAL — membuka aplikasi telepon/dial dengan nomor yang sudah terisi di layar panggilan, tetapi TIDAK langsung melakukan panggilan; pengguna sendiri yang harus menekan tombol panggil.

Intent.ACTION_SENDTO — mengarahkan ke aplikasi yang dapat mengirim pesan/data ke tujuan tertentu (misalnya email atau SMS) tanpa menyertakan lampiran/attachment file, biasanya digunakan untuk menyiapkan draft pesan/email dengan tujuan yang sudah terisi.

16. Apa fungsi data pada intent?

Field data pada Intent berfungsi menentukan objek atau konten spesifik yang akan diproses oleh action tersebut, biasanya dalam bentuk URI. Skema URI (seperti https:, tel:, mailto:, geo:) memberitahu sistem Android jenis konten/data yang dimaksud, sehingga sistem dapat mencari dan menawarkan aplikasi yang mampu menangani kombinasi action dan jenis data tersebut.

17. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android?

Action menyatakan "apa yang ingin dilakukan" (misalnya melihat, memanggil, atau mengirim), sedangkan Data menyatakan "objek/konten apa" yang menjadi sasaran aksi tersebut (URL, nomor telepon, alamat email, atau koordinat). Sistem Android mencocokkan kombinasi action dan skema data ini dengan intent-filter yang dideklarasikan oleh aplikasi-aplikasi yang terpasang; jika ada lebih dari satu aplikasi yang cocok dengan kombinasi tersebut, sistem akan menampilkan chooser (dialog pilihan aplikasi) agar pengguna memilih aplikasi mana yang ingin digunakan untuk menjalankan implicit intent tersebut.